

Spring Boot + H2 미니 게시판 만들기

static HTML + JavaScript fetch + REST API 방식

이 교재는 수업 중 바로 따라 치기 위한 코드 중심 자료입니다. 설명은 짧게, 확인은 자주 합니다.

사용하지 않는 것: templates, Thymeleaf, @Controller 화면 이동. 사용하는 것: static HTML, fetch(), @RestController, H2, JPA.

확인: Spring Console / Thunder Client / H2 Console / Browser Console

전체 진행 순서

목표: 작은 기능 하나를 만들고 바로 확인하면서 게시판 CRUD 를 완성한다.

01. 프로젝트 의존성 확인
02. H2 + JPA 설정
03. Board Entity 만들기
04. BoardRepository 만들기
05. BoardApiController 만들기
06. Thunder Client 로 API 먼저 확인
07. static 화면 구조 만들기
08. 글쓰기 페이지와 POST 저장
09. 목록 페이지와 GET 조회
10. 상세 페이지 이동과 GET 단건 조회
11. 수정 PUT
12. 삭제 DELETE
13. H2 Console 확인
14. 마무리 체크리스트

확인: 코드 작성 후 바로 실행 / API 는 Thunder 로 먼저 확인 / DB 는 H2 에서 SELECT / JS 는 console.log 로 확인

STEP 1. build.gradle 의존성 확인

목표: 웹, JPA, H2 를 사용할 수 있게 준비한다.

build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.4.3'
    id 'io.spring.dependency-management' version '1.1.7'
}

group = 'com.example'
version = '0.0.1-SNAPSHOT'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    runtimeOnly 'com.h2database:h2'

    compileOnly 'org.projectlombok:lombok'
    annotationProcessor 'org.projectlombok:lombok'

    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.named('test') {
    useJUnitPlatform()
}
```

developmentOnly 'org.springframework.boot:spring-boot-devtools' // 자동 재실행

Thymeleaf 를 쓰지 않는 방식이므로 spring-boot-starter-thymeleaf 는 필수가 아니다. 있어도 큰 문제는 없지만 이번 방식에서는 사용하지 않는다.

확인: Gradle 새로고침 / bootRun 실행 / 서버가 켜지는지 확인

STEP 2. application.properties 설정

목표: H2 메모리 DB 와 JPA SQL 로그를 켜다.

src/main/resources/application.properties

```
server.port=8080

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

JPA(Java Persistence API)는 자바 진영의 ORM(Object-Relational Mapping) 기술 표준입니다. 객체를 관계형 데이터베이스(RDB) 테이블에 직접 매핑하여, 개발자가 반복적인 SQL을 작성하지 않고도 객체 중심으로 데이터베이스를 관리할 수 있게

create 는 서버를 실행할 때마다 테이블을 새로 만든다. 수업용 H2 메모리 DB 에서는 이 설정이 가장 단순하다.

확인: Spring Console: HikariPool 시작 확인 / H2 Console: <http://localhost:8080/h2-console> / JDBC URL: jdbc:h2:mem:testdb

STEP 3. Board Entity 만들기

목표: 게시글 데이터가 저장될 테이블 구조를 만든다.

src/main/java/com/example/hello/Board.java

```
package com.example.hello;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
```

8-거

Spring Boot + H2 미니 게시판 실습 교재

```
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Getter
@Setter
@NoArgsConstructor
public class Board {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String writer;

    @Column(length = 1000)
    private String content;
}
```

✓ @Entity 는 DB 테이블로 만들 클래스라는 뜻이다. id 는 글 번호 / title/writer/content 는 게시글 내용이다.

확인: Spring Console: create table board 확인 / H2 Console: SELECT * FROM BOARD; 실행

STEP 4. BoardRepository 만들기

목표: DB 저장/조회/수정/삭제 기능을 간단히 사용한다.

src/main/java/com/example/hello/BoardRepository.java

```
package com.example.hello;

import org.springframework.data.jpa.repository.JpaRepository;

public interface BoardRepository extends JpaRepository<Board, Long> {
}
```

JpaRepository 를 상속하면 save(), findAll(), findById(), deleteById()를 바로 사용할 수 있다.

확인: 서버 재실행 / Spring Console: Found 1 JPA repository 확인

STEP 5. REST API Controller 만들기

목표: 브라우저 화면이 아닌 JSON 데이터를 주고받는 API 를 만든다.

src/main/java/com/example/hello/BoardApiController.java

```
package com.example.hello;

import java.util.List;

import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import lombok.RequiredArgsConstructor;
```

✓ import lombok.RequiredArgsConstructor;

✓ @RestController
@RequestMapping("/api/boards")

✓ @RequiredArgsConstructor

public class BoardApiController {

private final BoardRepository boardRepository;

조회
@GetMapping
public List<Board> list() {

return boardRepository.findAll();

}

등록
@PostMapping
public Board save(@RequestBody Board board) {

return boardRepository.save(board);

}

// 이 클래스는 화면을 보여주는 Controller가 아니라
// 데이터를 JSON으로 주고받는 API Controller이다.

// 이 컨트롤러의 기본 주소를 /api/boards 로 정한다.

// 모두 이 주소를 기준으로 동작 GET, POST, PUT, DELETE 요청.

// final 필드에 필요한 객체를 자동으로 넣어주는 생성자를 만들어준다.

// 여기서는 BoardRepository를 자동으로 연결해준다.

// DB 작업을 담당하는 Repository

// 게시글 저장, 조회, 수정, 삭제할 때 사용한다.

// 게시글 목록 조회 API

// 브라우저 또는 fetch에서 GET /api/boards 요청이 오면 실행됨

// DB에 저장된 모든 게시글을 조회해서 List 형태로 반환

// 반환된 데이터는 JSON 형태로 브라우저에 전달됨

// 게시글 저장 API

// fetch에서 POST /api/boards 요청이 오면 실행됨

// @RequestBody

// 프론트엔드에서 JSON으로 보낸 데이터를 Board 객체로 변환해 줌
// 예)

// {

// "title": "제목",

// "writer": "작성자",

// "content": "내용"

• // }

// 전달받은 Board 객체를 DB에 저장

// save() 실행 후 저장된 Board 객체를 다시 반환

Spring Boot + H2 미니 게시판 실습 교재

```

@GetMapping("/{id}") 상세조회는 CRUD중예?
public Board detail(@PathVariable Long id) {
    return boardRepository.findById(id).orElseThrow();
}

@PutMapping("/{id}") 수정
public Board update(@PathVariable Long id, @RequestBody Board data) {
    Board board = boardRepository.findById(id).orElseThrow();
    board.setTitle(data.getTitle());
    board.setWriter(data.getWriter());
    board.setContent(data.getContent());
    return boardRepository.save(board);
}

@DeleteMapping("/{id}") 삭제
public void delete(@PathVariable Long id) {
    boardRepository.deleteById(id);
}
}

```

@RestController 는 문자열/객체를 JSON 응답으로 보낸다. 화면 이동용 @Controller 와 역할이 다르다.

확인: 서버 재실행 / 중복 매핑 에러 없는지 확인 / Thunder Client 로 다음 단계 진행

STEP 6. Thunder Client 로 API 먼저 확인

목표: HTML 을 만들기 전에 서버 API 가 정상인지 먼저 확인한다.

목록 조회 테스트

```
GET http://localhost:8080/api/boards
```

```
정상 예시:
[]
```

저장 테스트

```
POST http://localhost:8080/api/boards
Header: Content-Type: application/json
```

```
Body(JSON):
{
  "title": "테스트 제목",
  "writer": "홍길동",
  "content": "테스트 내용"
}
```

```
정상 예시:
{
  "id": 1,
  "title": "테스트 제목",
  "writer": "홍길동",
  "content": "테스트 내용"
}
```

상세/수정/삭제 테스트 주소

```

GET http://localhost:8080/api/boards/1
PUT http://localhost:8080/api/boards/1
DELETE http://localhost:8080/api/boards/1

```

확인: Thunder: POST 성공 / H2: SELECT * FROM BOARD; / Spring Console: insert/select SQL 확인

STEP 7. static 화면 폴더 만들기

목표: 일반 HTML 파일을 static 폴더에 둔다.

이번 수업의 화면 파일 구조

```

src/main/resources
├── static
│   ├── index.html
│   ├── about.html
│   └── board
│       ├── list.html
│       ├── write.html
│       └── detail.html

```

templates 는 Thymeleaf 용이다. 이번 방식은 static HTML + fetch 이므로 templates 를 사용하지 않는다.

Spring Boot + H2 미니 게시판 실습 교재

확인: <http://localhost:8080/index.html> / <http://localhost:8080/board/write.html> / HTML 파일이 직접 열리는지 확인

STEP 8. index.html 만들기

목표: 홈 화면과 메뉴를 만든다.

src/main/resources/static/index.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>미니 게시판</title>
  <style>
    body { font-family: Arial; background: #f5f6fa; margin: 0; }
    header { background: #333; color: white; padding: 30px; text-align: center; }
    nav { background: white; padding: 15px; text-align: center; border-bottom: 1px solid #ddd; }
    nav a { color: #333; text-decoration: none; margin: 0 15px; font-weight: bold; }
    main { width: 800px; margin: 40px auto; background: white; padding: 30px; border-radius: 10px; }
    .btn { display: inline-block; background: #333; color: white; padding: 10px 20px; text-decoration: none; border-radius: 5px; }
  </style>
</head>
<body>
  <header>
    <h1>우리 반 미니 게시판</h1>
    <p>Spring Boot + H2 + REST API + fetch</p>
  </header>

  <nav>
    <a href="/">홈</a>
    <a href="/about.html">소개</a>
    <a href="/board/list.html">게시판</a>
    <a href="/board/write.html">글쓰기</a>
  </nav>

  <main>
    <h2>환영합니다</h2>
    <p>HTML 화면에서 JavaScript fetch로 Spring Boot API를 호출합니다.</p>
    <a class="btn" href="/board/list.html">게시판 보러가기</a>
  </main>
</body>
</html>
```

확인: 브라우저: <http://localhost:8080/> / 메뉴 클릭으로 페이지 이동 확인

STEP 9. about.html 만들기

목표: 간단한 소개 페이지를 만든다.

src/main/resources/static/about.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>소개</title>
  <style>
    body { font-family: Arial; background: #f5f6fa; margin: 0; }
    nav { background: #333; padding: 15px; text-align: center; }
    nav a { color: white; text-decoration: none; margin: 0 15px; }
    main { width: 800px; margin: 40px auto; background: white; padding: 30px; border-radius: 10px; }
  </style>
</head>
<body>
  <nav>
    <a href="/">홈</a>
    <a href="/about.html">소개</a>
    <a href="/board/list.html">게시판</a>
    <a href="/board/write.html">글쓰기</a>
  </nav>

  <main>
    <h1>소개 페이지</h1>
    <p>이 사이트는 수업 실습용 미니 게시판입니다.</p>
  </main>
</body>
</html>
```

확인: 브라우저: </about.html> / 메뉴 이동 확인

STEP 10. write.html 화면 만들기

목표: 제목, 작성자, 내용을 입력할 화면을 만든다.

src/main/resources/static/board/write.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>글쓰기</title>
  <style>
    body { font-family: Arial; background: #f5f6fa; margin: 0; }
    nav { background: #333; padding: 15px; text-align: center; }
    nav a { color: white; text-decoration: none; margin: 0 15px; }
    main { width: 800px; margin: 40px auto; background: white; padding: 30px; border-radius: 10px; }
    input, textarea { width: 100%; padding: 10px; margin: 10px 0; box-sizing: border-box; }
    button { background: #333; color: white; padding: 10px 25px; border: none; border-radius: 5px; cursor: pointer; }
  </style>
</head>
<body>
  <nav>
    <a href="/">홈</a>
    <a href="/about.html">소개</a>
    <a href="/board/list.html">게시판</a>
    <a href="/board/write.html">글쓰기</a>
  </nav>

  <main>
    <h1>글쓰기</h1>

    <input type="text" id="input-title" placeholder="제목을 입력하세요">
    <input type="text" id="input-writer" placeholder="작성자를 입력하세요">
    <textarea rows="8" id="input-content" placeholder="내용을 입력하세요"></textarea>

    <button type="button" onclick="createPost()">등록하기</button>
  </main>

  <script>
    function createPost() {
      alert("아직 저장 기능을 연결하지 않았습니다.");
    }
  </script>
</body>
</html>
```

확인: 브라우저: /board/write.html / 등록 버튼 클릭 시 alert 확인

STEP 11. write.html 에 저장 기능 붙이기

목표: 입력값을 JSON 으로 만들어 POST 요청을 보낸다.

write.html 의 기존 <script> 부분 교체

```
<script>
function createPost() {
  const title = document.getElementById("input-title").value;
  const writer = document.getElementById("input-writer").value;
  const content = document.getElementById("input-content").value;

  if (!title || !writer || !content) {
    alert("제목, 작성자, 내용을 모두 입력하세요.");
    return;
  }

  const postData = {
    title: title,
    writer: writer,
    content: content
  };

  console.log("저장할 데이터:", postData);

  fetch("/api/boards", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(postData)
  })
  .then(response => {
    if (response.ok) {
      alert("등록 완료");
      location.href = "/board/list.html";
    } else {

```

```

    alert("등록 실패");
  }
});
}
</script>

```

button에는 onclick="createPost()"처럼 괄호가 있어야 함수가 실행된다.

확인: Browser Console: `console.log(postData)` / Thunder: `GET /api/boards` / H2: `SELECT * FROM BOARD;` / Spring Console: insert SQL 터미널 창에 찍히는지...

STEP 12. list.html 화면 만들기

목표: 게시글 목록을 보여줄 테이블을 만든다.

src/main/resources/static/board/list.html

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>게시판</title>
  <style>
    body { font-family: Arial; background: #f5f6fa; margin: 0; }
    nav { background: #333; padding: 15px; text-align: center; }
    nav a { color: white; text-decoration: none; margin: 0 15px; }
    main { width: 800px; margin: 40px auto; background: white; padding: 30px; border-radius: 10px; }
    table { width: 100%; border-collapse: collapse; }
    th, td { border-bottom: 1px solid #ddd; padding: 12px; text-align: center; }
    th { background: #f0f0f0; }
    .title { cursor: pointer; color: #0066cc; font-weight: bold; }
    .btn { display: inline-block; margin-top: 20px; background: #333; color: white; padding: 10px 20px; text-decoration: none; border-radius: 5px; }
  </style>
</head>
<body>
  <nav>
    <a href="/">홈</a>
    <a href="/about.html">소개</a>
    <a href="/board/list.html">게시판</a>
    <a href="/board/write.html">글쓰기</a>
  </nav>

  <main>
    <h1>게시판 목록</h1>

    <table>
      <thead>
        <tr>
          <th>번호</th>
          <th>제목</th>
          <th>작성자</th>
        </tr>
      </thead>
      <tbody id="board-list">
        <tr>
          <td colspan="3">게시글을 불러오는 중...</td>
        </tr>
      </tbody>
    </table>

    <a class="btn" href="/board/write.html">글쓰기</a>
  </main>

  <script>
    fetchBoards();

    function fetchBoards() {
      fetch("/api/boards")
        .then(response => response.json())
        .then(data => {
          console.log("서버에서 받은 목록:", data);

          const list = document.getElementById("board-list");
          list.innerHTML = "";

          if (data.length === 0) {
            list.innerHTML = '<tr><td colspan="3">등록된 글이 없습니다.</td></tr>';
            return;
          }

          data.forEach(board => {
            const row = document.createElement("tr");

            row.innerHTML = `
              <td>${board.id}</td>
              <td class="title" onclick="goDetail(${board.id})">${board.title}</td>
            `;
          });
        });
    }
  </script>

```



```

        <td>${board.writer}</td>
    </tr>
    </tbody>
</table>

    list.appendChild(row);
});
});
}

function goDetail(id) {
    location.href = "/board/detail.html?id=" + id;
}
</script>
</body>
</html>

```

table 안에는 ~~div가 아니라~~ tr, td를 넣는다. tbody에 id를 주고 JavaScript로 tr을 추가한다.

확인: Browser Console: console.log(data) / 브라우저: 목록 출력 확인 / 제목 클릭: detail.html?id=번호 이동

STEP 13. detail.html 상세조회 만들기

목표: 주소의 id를 읽어서 해당 게시글을 조회한다.

src/main/resources/static/board/detail.html

```

<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <title>상세보기</title>
    <style>
        body { font-family: Arial; background: #f5f6fa; margin: 0; }
        nav { background: #333; padding: 15px; text-align: center; }
        nav a { color: white; text-decoration: none; margin: 0 15px; }
        main { width: 800px; margin: 40px auto; background: white; padding: 30px; border-radius: 10px; }
        input, textarea { width: 100%; padding: 10px; margin: 10px 0; box-sizing: border-box; }
        button, .btn { display: inline-block; background: #333; color: white; padding: 10px 18px; border: none; border-radius: 5px; text-
        decoration: none; cursor: pointer; }
        .edit { background: #198754; }
        .delete { background: #dc3545; }
    </style>
</head>
<body>
    <nav>
        <a href="/">홈</a>
        <a href="/about.html">소개</a>
        <a href="/board/list.html">게시판</a>
        <a href="/board/write.html">글쓰기</a>
    </nav>

    <main>
        <h1>게시글 상세보기</h1>

        <p>번호: <span id="detail-id"></span></p>

        <label>제목</label>
        <input type="text" id="detail-title">

        <label>작성자</label>
        <input type="text" id="detail-writer">

        <label>내용</label>
        <textarea id="detail-content" rows="8"></textarea>

        <button class="edit" onclick="updateBoard()">수정하기</button>
        <button class="delete" onclick="deleteBoard()">삭제하기</button>
        <a class="btn" href="/board/list.html">목록으로</a>
    </main>

    <script>
        const params = new URLSearchParams(location.search);
        const id = params.get("id");

        console.log("상세 조회 id:", id);

        fetch("/api/boards/" + id)
            .then(response => response.json())
            .then(board => {
                console.log("상세 데이터:", board);

                document.getElementById("detail-id").innerText = board.id;
                document.getElementById("detail-title").value = board.title;
                document.getElementById("detail-writer").value = board.writer;
                document.getElementById("detail-content").value = board.content;
            });
    </script>

```

```

function updateBoard() {
    alert("다음 단계에서 수정 기능을 연결합니다.");
}

function deleteBoard() {
    alert("다음 단계에서 삭제 기능을 연결합니다.");
}
</script>
</body>
</html>

```

list.html 에서 /board/detail.html?id=3 으로 이동하면 detail.html 에서 id=3 을 꺼내 API 를 호출한다.

확인: Browser Console: console.log(id) / Browser Console: console.log(board) / Thunder: GET /api/boards/1

STEP 14. 수정 기능 붙이기 백엔드 추가 5page

목표: 상세 화면의 입력값을 PUT 요청으로 보낸다.

✓ detail.html 의 기존 updateBoard() 함수 교체

```

function updateBoard() {
    const title = document.getElementById("detail-title").value;
    const writer = document.getElementById("detail-writer").value;
    const content = document.getElementById("detail-content").value;

    const updateData = {
        title: title,
        writer: writer,
        content: content
    };

    console.log("수정할 데이터:", updateData);

    fetch("/api/boards/" + id, {
        method: "PUT",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(updateData)
    })
    .then(response => {
        if (response.ok) {
            alert("수정 완료");
            location.href = "/board/list.html";
        } else {
            alert("수정 실패");
        }
    })
    });
}

```

확인: Browser Console: console.log(updateData) / Spring Console: update SQL / H2: SELECT * FROM BOARD; / 목록에서 수정 내용 확인

STEP 15. 삭제 기능 붙이기

목표: DELETE 요청으로 게시글을 삭제한다.

✓ detail.html 의 기존 deleteBoard() 함수 교체

```

function deleteBoard() {
    if (!confirm("정말 삭제하시겠습니까?")) {
        return;
    }

    console.log("삭제할 id:", id);

    fetch("/api/boards/" + id, {
        method: "DELETE"
    })
    .then(response => {
        if (response.ok) {
            alert("삭제 완료");
            location.href = "/board/list.html";
        } else {
            alert("삭제 실패");
        }
    })
    });
}

```

확인: Browser Console: console.log(id) / Spring Console: delete SQL / H2: SELECT * FROM BOARD; / 목록에서 삭제 확인

Spring Boot + H2 미니 게시판 실습 교재

STEP 16. H2 Console 확인 방법

목표: 웹 화면에서 입력한 데이터가 DB에 실제 저장됐는지 확인한다.

H2 Console 접속 정보

```
주소:
http://localhost:8080/h2-console

JDBC URL:
jdbc:h2:mem:testdb

User Name:
sa

Password:
비워둬
```

게시글 전체 조회

```
SELECT * FROM BOARD;
```

H2에서 직접 데이터 넣기

```
INSERT INTO BOARD(TITLE, WRITER, CONTENT)
VALUES('H2에서 추가한 글', '관리자', 'DB에서 직접 추가한 내용입니다.');
```

확인: H2에서 INSERT 실행 / 브라우저 /board/list.html 새로고침 / 직접 넣은 글이 보이는지 확인

STEP 17. Thunder Client 확인 목록

목표: 화면 연결 전후로 API가 정상인지 빠르게 점검한다.

Thunder Client 테스트 주소 모음

```
1. 목록 조회
GET http://localhost:8080/api/boards

2. 글 저장
POST http://localhost:8080/api/boards
Content-Type: application/json
{
  "title": "썬더 제목",
  "writer": "썬더 작성자",
  "content": "썬더 내용"
}

3. 상세 조회
GET http://localhost:8080/api/boards/1

4. 수정
PUT http://localhost:8080/api/boards/1
Content-Type: application/json
{
  "title": "수정 제목",
  "writer": "수정 작성자",
  "content": "수정 내용"
}

5. 삭제
DELETE http://localhost:8080/api/boards/1
```

확인: API 실패 시 HTML보다 Controller부터 확인 / API 성공인데 화면 실패 시 JS console 확인

STEP 18. 자주 나는 문제 체크

목표: 에러가 났을 때 어디를 봐야 하는지 빠르게 찾는다.

빠른 점검표

```
증상: 등록 버튼을 눌러도 아무 일도 안 일어남
확인: onclick="createPost()" 괄호가 있는지 확인

증상: JS에서 null 에러
확인: getElementById("input-title")와 HTML id가 같은지 확인

증상: 목록이 안 나옴
확인: Browser Console에 console.log(data) 찍기

증상: table이 깨짐
```

Spring Boot + H2 미니 게시판 실습 교재

확인: table 안에는 div 가 아니라 tbody/tr/td 사용

증상: API 는 되는데 화면만 안 됨

확인: fetch 주소 /api/board **s** 오타 확인

증상: 서버 실행 실패

확인: 같은 URL 을 두 메서드가 잡고 있지 않은지 확인

증상: H2 에 테이블 없음

확인: spring.jpa.hibernate.ddl-auto=create 확인

확인: Spring Console / Browser Console / Thunder Client / H2 Console 순서로 확인

STEP 19. 최종 동작 흐름

목표: 사용자가 누른 버튼이 DB 까지 연결되는 흐름을 정리한다.

CRUD 전체 흐름

글쓰기 저장 흐름

write.html 입력
→ createPost()
→ fetch POST /api/boards
→ BoardApiController.save()
→ boardRepository.save()
→ H2 BOARD 테이블 저장
→ list.html 이동

목록 조회 흐름

list.html 열림
→ fetch GET /api/boards
→ BoardApiController.list()
→ boardRepository.findAll()
→ JSON 응답
→ JavaScript가 table 에 tr 추가

상세 조회 흐름

목록에서 제목 클릭
→ /board/detail.html?id=글번호
→ URLSearchParams 로 id 꺼냄
→ fetch GET /api/boards/id
→ 상세 데이터 출력

수정 흐름

detail.html 입력값 수정
→ updateBoard()
→ fetch PUT /api/boards/id
→ DB 수정
→ 목록 이동

삭제 흐름

detail.html 삭제 버튼
→ deleteBoard()
→ fetch DELETE /api/boards/id
→ DB 삭제
→ 목록 이동

확인: Create 저장 / Read 목록/상세 / Update 수정 / Delete 삭제 / H2 에서 최종 확인

STEP 20. 수업 진행용 짧은 순서표

목표: 수업 중 순서를 잊지 않기 위한 진행 메모다.

강사용 진행 체크리스트

1. 최종 화면 먼저 보여주기
2. build.gradle 확인
3. application.properties 작성
4. Board Entity 작성
5. BoardRepository 작성
6. BoardApiController 작성
7. Thunder GET/POST 확인
8. H2 SELECT 확인
9. index/about 페이지 확인
10. write.html 작성
11. createPost() 붙이기
12. Browser Console 로 postData 확인
13. list.html 작성
14. console.log(data) 확인
15. detail.html 작성
16. console.log(id), console.log(board) 확인
17. updateBoard() 추가

Spring Boot + H2 미니 게시판 실습 교재

```
18. deleteBoard() 추가
19. H2 에서 최종 SELECT
20. CRUD 흐름 정리
```

각 기능마다 코드를 붙인 직후 바로 확인한다. 확인 루틴을 자주 반복하는 것이 핵심이다.

확인: 코드 80% / 설명 20% / 확인은 자주 / 에러는 콘솔에서 먼저 찾기